Lab02 – SLURM part 1

Introduction

The objective of this exercise is to run a Jupyter notebook as an interactive SLURM job on the Ares or Athena cluster.

Useful links:

1. Ares
2. Athena
3. Slurm Workload Manager - Quick Start User Guide
4. Podstawy SLURM
5. Ustawianie środowiska aplikacji - Modules

SLURM jobs

A SLURM job is created from a shell script that typically has the following structure:

```
#!/bin/bash
#SBATCH --ntasks=1
#SBATCH --time=00:05:00
#SBATCH --cpus-per-task=4
#SBATCH -p plgrid
# the above lines are read and interpreted by sbatch, this one and later are not

# initialization
module load package/version

# data handling
cd /path/to/working/directory

# job command - this will be executed on a compute node
sleep 1m

exit
```

The job is submitted using the **sbatch** command.

A SLURM job is a collection of **tasks**. A task exists on a single node, but may consist of multiple processes (e.g. forked).

A job may also consist of multiple **steps** which can run sequentially or in parallel. Steps are specified using **srun** command inside a job script. An example SLURM job with steps is shown in Fig. 1.

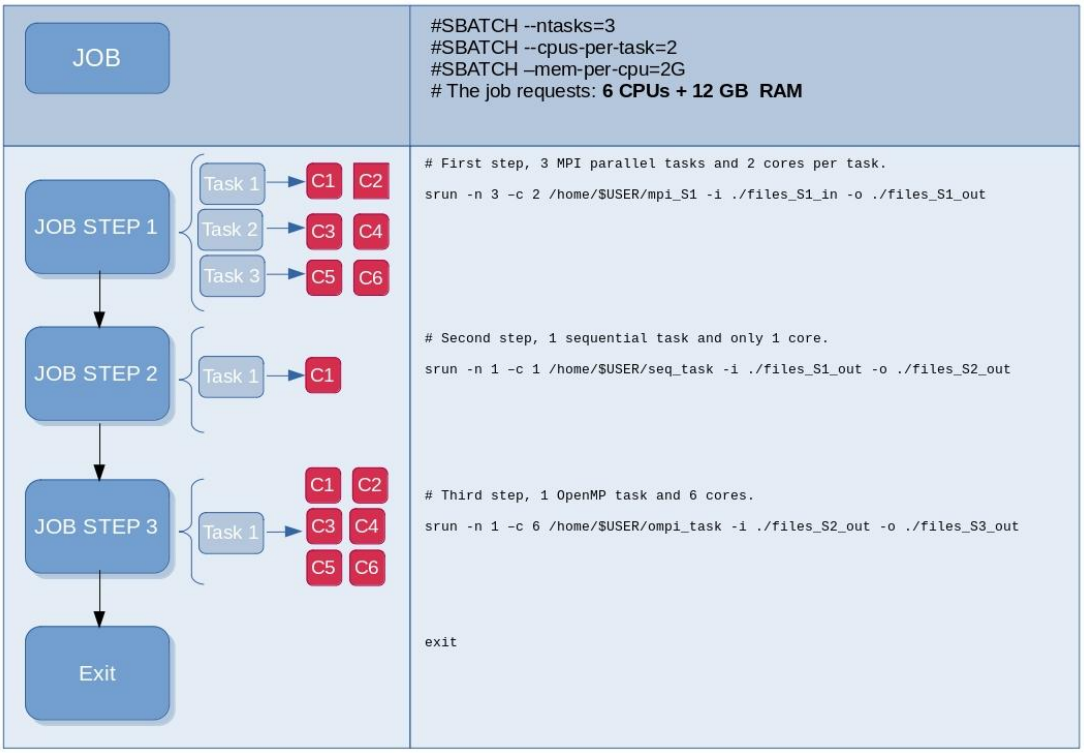


Figure 1: Example SLURM job structure and script. (Source: https://garnatxadoc.uv.es/slurm/slurm_info.html)

SLURM architecture

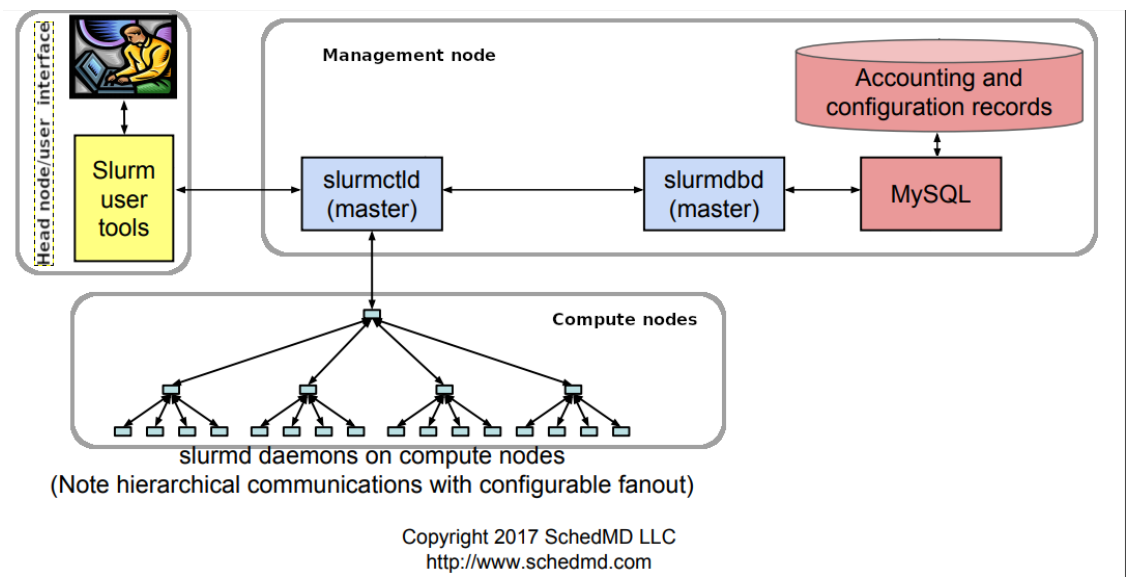


Figure 2: SLURM architecture

SLURM entities

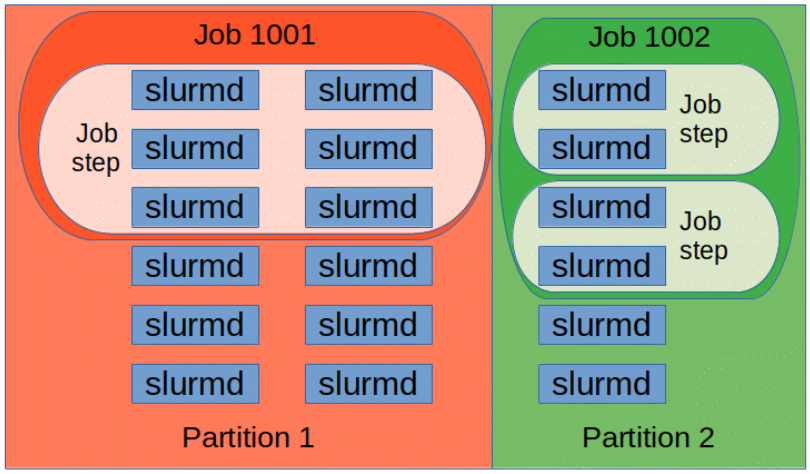


Figure 3: SLURM entities

SLURM commands

Basic commands

- sinfo – info about nodes
- squeue – queue status
- sbatch – submit a job
- srun – run an interactive job/job step
- scancel – cancel job
- sacct – accounting information

Examples:

```
sinfo -p plgrid
squeue
sbatch job.sh
srun -N 1 --ntasks-per-node=1 -p plgrid-testing --pty /bin/bash -l
scancel <jobid>
sacct -X
```

Resource specification

- `sbatch -N 2` – Node count
- `sbatch -N 1 --ntasks-per-node=6` – Node count with 6 tasks per node = 6 cores
- `sbatch -N 2 --ntasks-per-node=6` – Node count with 6 tasks per node = 12 cores on 2 nodes
- `sbatch -N 2 --ntasks-per-node=6 -t 20:00` – declared duration of job (HH:MM:SS), defaults to 15 minutes
- `sbatch -A plglscclass24-cpu` – declare using the plglscclass24-cpu grant for your computations, this is important if you have other computing grants and want to use a specific one!

- The account name plglscclass24-cpu can be determined by inspecting the `hpc-grants` output, and looking for "allocation" name. Please note that there can be multiple grants, use the most recent one!
- Be cautious about available node configurations, you are responsible for "fitting in"

Interactive jobs

```
srun -p plgrid-now --time 1:00:00 --pty /bin/bash -l
```

- Hint: using the plgrid-now partition usualy leads to very short queue wait time with the cost of imposing limits on the job (short runtime)
- Hint2: using the plgrid-testing partition also yelds shorter queue times with less severe restrictions

Environment

SLURM communicates a lot of information through environment variables

- $SLURM_SUBMIT_HOST - host, what is the host from which the job originated?
- $SLURM_SUBMIT_DIR – directory, from where the job was submitted
- Other variables: man sbatch, or check with 'env' command in an interactive job

Array/parametric jobs

We can create multiple instances of the same job by using the `–array` parameter.

```
Example:
sbatch -N 1 --ntasks-per-node=1 -p plgrid-testing --array=1-4 test.sh
```

This will create an array job composed of 4 jobs which can be verified by using the squeue command. Each job has a unique identifier which can be obtained by reading the SLURM_ARRAY_TASK_ID environment variable.

```
echo $SLURM_ARRAsY_TASK_ID
```

Modules

Managing available software is done through the system called "modules".

- The software includes: scientific software, libraries (e.g. sqlite, cuda), core utilities (compiles, math libraries)
- The "module" command is used to work with the system, where subcommands include:
  - avail – see the list of availble modules
  - load – to load a module
  - list – to list loaded modules
  - purge – clean your environment from any laoded modules
  - help – consult the manual
- Examples:
  `module avail`
  (..This will list all modules..)
  `module load plumed`
  (..This will load the plumed module..)
  `module purge`
  (..This will list the unloaded modules..)

- The module command doesn’t install the software, it adjusts the environment, so the application becomes available as it would be installed in a usual way. You can think of a module as a preset of PATH, LD_LIBRARY_PATH and other important environment variables.
- Load modules in the job script!

Exercise - running Jupyter on a supercomputer – step-by-step

Important!

Remember how to use different storage (consult Ares and Athena documentation).

Location	Location in the filesystem	Purpose
$HOME	/net/people/plgrid/<login>	Storing own applications, and configuration files. Limited to 10GB.
$SCRATCH	/net/afscra/people/<login>	High-speed storage for short-lived data used in computations. Data older than 30 days can be deleted without notice. It is best to rely on the $SCRATCH environment variable.
$PLG_GROUPS_STORAGE/<group name>	/net/pr2/projects/plgrid/<group name>	Shared group storage. Long-term storage for data living for the period of computing grant. Should be used for storing significant amounts of data. Uses transparent compression.

Do not alter existing data on the group storage space! For example, when installing python packages, make sure you switch to your private virtual evironment!

Note: what to pass to **--partition** and **--account**?

- To see available partitions, use command **sinfo -s**
- For the account, use command **hpc-grants** (on login node) and look for different "Allocation" entries.
- For example, when using GPU you should choose the appropriate partition with GPUs, and Allocation associated with the GPU resource.
- Example **hpc-grants** output on Ares:

```
Grant: plglscclass24
  status: active, start: 2024-10-01, end: 2025-09-30
```

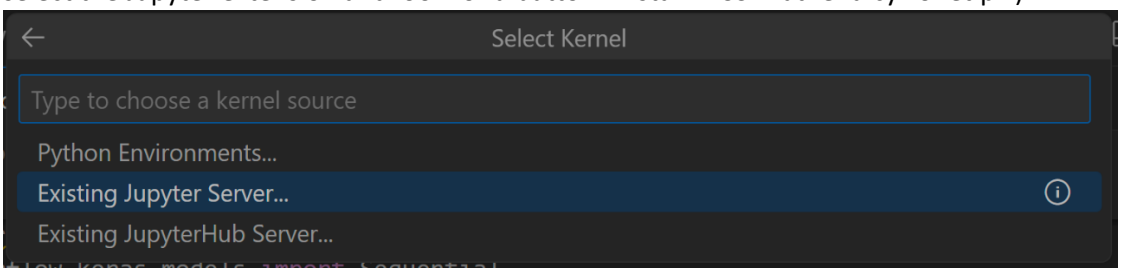
```
Allocation: plglscclass24-cpu, resource: cpu
 status: active, start: 2024-10-01, end: 2025-09-30,
 parameters: hours: 1 000 000 h, timelimit: 72 h
 consumed resources: cpu: 0.50 h
Allocation: plglscclass24-gpu, resource: gpu
 status: active, start: 2024-10-01, end: 2025-09-30,
 parameters: hours: 1 000 h
 consumed resources: gres/gpu: 16.85 h
Allocation: plglscclass24, resource: storage
 status: active, start: 2024-10-01, end: 2025-09-30,
 parameters: capacity: 100 GB
Group: plgglscclass
```

Steps to do on Ares/Athena:

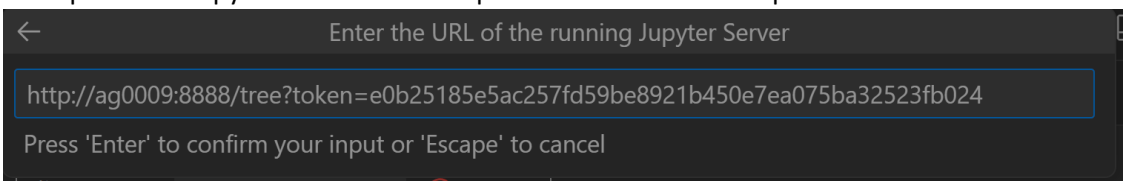
1. Login to Ares or Athena (the lab was tested on Ares)
2. Run an interactive job using command:
   Ares:
   srun --time=2:00:00 --mem=8G --ntasks 1 --gres=gpu:1 --partition=plgrid-gpu-v100 --account=plglscclass26-gpu --pty /bin/bash
   Athena:
   srun --time=2:00:00 --mem=8G --ntasks 1 --gres=gpu:1 --partition=plgrid-gpu-a100 --account =plglscclass26-gpu-a100 --pty /bin/bash
   You will see output similar to:
   srun: job 11818449 queued and waiting for resources
   srun: job 11818449 has been allocated resources
   [ares][plgbalis@ag0009 ~]$
   This interactive job provides a shell access to a worker node, in this case **ag0009**.
   Note: this job requests a GPU, so it may take a longer time to start. In extreme cases all GPUs could be allocated by different jobs.
3. Activate a python virtual environment prepared on the shared group storage:
   conda activate $PLG_GROUPS_STORAGE/plgglscclass/.conda/envs/tf2-gpu
   - If conda is not available, enable it by using **module load miniconda3** (Miniconda3 on Athena)
   - The first time you run conda it might be necessary to initialize it with command **conda init bash** (after which the shell needs to be reloaded)
   - You can look at installed packages using **conda list**
4. Copy example notebook to your home directory:
   cp $PLG_GROUPS_STORAGE/plgglscclass/lsc_lab02.ipynb $HOME
5. Start jupyter (provide your node name to **--ip** parameter):
   jupyter notebook --no-browser --port=<port> --ip=<node_name>
   - Use a random port number (higher than 1024) to avoid port conflicts
   - <node_name> is the name of the node you should see in the prompt, e.g. ag0009
   - Note the connection URL, it will be needed later!

Steps to do on your machine:

6. Start Visual Studio Code and install extension **Remote – SSH** from Microsoft.
7. Start a remote SSH connection to Ares/Athena (you can invoke command palette **Ctrl+Shift+P** and search for **Remote-SSH: Connect to Host**)
8. Open the **lsc_lab02.ipynb** file
9. Connect to the Jupyter server started on the worker node.
   First choose "Existing Jupyter Server"
   Note: you might need to install Jupyter extension on the remote server first (go to marketplace, select the Jupyter extension and look for a button "Install in SSH: athena.cyfronet.pl")



   Then paste the Jupyter connection URL printed earlier. For example:



10. Run the example code in the first input cell and see if it works!

Assignment

1. (3 pts) Configure a new python environment on your SCRATCH space and run the second example provided in the **lsc_lab02.ipynb** notebook. The example compares matrix multiplication performance on CPU and GPU. Run it a few times and report obtained results. Increase the matrix size and repeat experiments.

   Use an AI chat (grok, chatgpt, gemini) to help you with the exercise: plan the experiment (matrix sizes) and interpret the results. Always verify output produced by AI in external sources. In your report, link the conversation with the chat.

2. (5 pts) Perform data analysis using Jupyter/Pandas/Matplotlib on the **Yelp dataset**, available in the shared group storage:

$PLG_GROUPS_STORAGE/plgglscclass/yelp_dataset

Configure an environment (python virtenv) on your SCRATCH storage.

Important: if you don't need GPU, use the **plgrid-now** partition to get a fast node allocation.

The dataset was downloaded from Yelp Dataset (kaggle.com). You can use notebooks available there.

Grading:

- Configuration of the environment: 2p
- Data analysis: 3p

Submission

Submit notebooks used in both tasks (in **ipynb** and – importantly – **html** formats). Include any additional information as comments or text cells. e.g.:

- commands used to configure the environments
- any remarks on the achieved results and encountered problems

Notes:

- It is better to configure your application (e.g. install packages) on a worker node using an interactive job, rather than on the login node
- To install your own packages, you can use your **$HOME** disk, but the quota is limited.
- You can also use your **$SCRATCH** storage space. To do this, you can configure conda:
  conda config --add envs_dirs ${SCRATCH}/.conda/envs
  conda config --add pkgs_dirs ${SCRATCH}/.conda/pkgs

Appendix: Jupyter code

```
import tensorflow as tf
from keras.datasets import mnist
from keras.models import Sequential
from keras.layers import Dense, Flatten

# Load and preprocess the data
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

physical_devices = tf.config.experimental.list_physical_devices('GPU')
print(physical_devices)

# Define the model
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=5)

# Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print('\nTest accuracy:', test_acc)
```

```
import numpy as np
import cupy as cp
import time

# Matrix size
N = 10000

# CPU-based matrix multiplication using NumPy
start_time = time.time()
A_cpu = np.random.rand(N, N)
B_cpu = np.random.rand(N, N)
C_cpu = np.dot(A_cpu, B_cpu)
```

```python
print(C_cpu[1][1])
cpu_time = time.time() - start_time
print(f"CPU time: {cpu_time:.2f} seconds")

# GPU-based matrix multiplication using CuPy
start_time = time.time()
A_gpu = cp.random.rand(N, N, dtype=cp.float32)
B_gpu = cp.random.rand(N, N, dtype=cp.float32)
C_gpu = cp.dot(A_gpu, B_gpu)
print(C_gpu[1][1])
gpu_time = time.time() - start_time
print(f"GPU time: {gpu_time:.2f} seconds")
```